

Guide d'étude express : le tidyverse

L'essentiel · la version complète détaille chaque notion et où la travailler

Évalué **en présence et sans IA** (évaluation sur table, oraux). On vérifie que vous **comprenez** une chaîne de commandes : prédire son résultat (souvent le **nombre de lignes**), expliquer pourquoi il est faux, anticiper l'effet d'une modification.

Savoir faire

Notion	Ce qu'il faut savoir faire
Importer	<code>read_csv()</code> / <code>read_tsv()</code> ; lire le compte rendu ; repérer une colonne de nombres lue en texte (<code><chr></code>)
Pipe	réécrire <code>f(g(x))</code> avec le <i>pipe</i> (<code>x</code> puis <code>g()</code> puis <code>f()</code>), et lire une chaîne « puis... puis... »
Verbes	<code>select()</code> , <code>filter()</code> , <code>mutate()</code> , <code>arrange()</code> , <code>count()</code> , <code>slice_max()</code> , <code>pull()</code> ; assigner avec <code><-</code> pour garder le résultat
Nettoyer	<code>str_to_lower()</code> , <code>str_trim()</code> , <code>distinct()</code> ; repérer les variantes et les doublons avec <code>count()</code>
Résumer	<code>group_by()</code> + <code>summarise()</code> (une ligne par groupe) vs <code>mutate()</code> (autant de lignes) ; <code>n()</code> vs <code>sum(n)</code>
Tidy	une variable par colonne, une observation par ligne, une valeur par case ; <code>pivot_longer()</code> / <code>pivot_wider()</code> et les dimensions obtenues
Joindre	<code>left_join()</code> , <code>inner_join()</code> , <code>full_join()</code> , <code>anti_join()</code> ; préciser <code>by</code>
Traduire	passer d'une commande en R de base à son équivalent <i>tidyverse</i>

Les pièges silencieux à connaître

Aucun ne produit d'erreur. Sachez les **prédire** et dire comment les **vérifier**.

1. `filter()` écarte les lignes où la condition vaut `NA` → vérifier que les parties font le tout.
2. `filter(x == 2019 | 2020)` garde **tout**; `filter(x == c("a", "b"))` **recycle** → écrire `x %in% c(...)`.
3. Après `summarise()` à deux groupes, le résultat **reste groupé** → un `sum()` suivant se fait par groupe.
4. `count()` compte des **lignes** : se demander ce que représente une ligne.
5. Une **clé en double** à droite d'un `left_join()` **ajoute** des lignes ; une clé mal saisie donne des `NA` → `nrow()` avant/après, `count(cle) |> filter(n > 1)`, `anti_join()`.
6. `Controle`, `controle` et `controle_` sont trois catégories → `count()` puis nettoyer.

Comment vous préparer

- Refaites **sans ordinateur** les pauses « À vous », les « Et si... ? » des TP et l'exercice formatif ; vérifiez ensuite dans RStudio.
- Quand une prédiction est fausse, **cherchez pourquoi** : c'est ce qu'on vous demandera à l'oral.
- Lisez les **explications** des solutionnaires, pas seulement les réponses.

